

TALLER ABP • ENTREGA 1

EcoRuta

*Reporte ciudadano y priorización automática
de puntos críticos de residuos sólidos*

App móvil Flutter • Arquitectura serverless en AWS • Infraestructura como código

Diego Gamba

Diseño de Aplicaciones Móviles • Facultad de Ingeniería, Ciencias y Administración

Los puntos críticos aparecen, se erradican y reaparecen

Parte de los residuos sólidos urbanos no llega al sistema formal de recolección: se acumula en esquinas, separadores y orillas de quebrada. El fenómeno tiene tres rasgos que lo hacen difícil de atender con los instrumentos actuales.

1

Es dinámico

Un punto se erradica y reaparece a media cuadra. Los inventarios manuales quedan obsoletos en semanas.

2

Es local

Quien mejor lo detecta es el vecino que pasa a diario, no el operador que recorre la macro-ruta.

3

No fluye la información

Los canales actuales producen texto libre sin coordenadas ni evidencia: obligan a verificar en campo.

Consecuencia operativa: recolección reactiva y no optimizada, riesgo sanitario por vectores, obstrucción de sumideros y deterioro del espacio público.

Qué nos preguntamos y qué nos propusimos

PREGUNTA QUE GUÍA EL PROYECTO

¿Puede un flujo de reporte ciudadano georreferenciado, con evidencia fotográfica y agrupamiento espacial automático, producir un inventario de puntos críticos más actualizado y accionable que el levantamiento manual periódico?

Objetivos específicos

- 1 Capturar un reporte georreferenciado con evidencia en menos de 60 segundos, incluso sin conectividad.
- 2 Convertir reportes dispersos en puntos críticos mediante agrupamiento espacial y priorizarlos por severidad.
- 3 Sugerir el tipo de residuo desde la fotografía para estandarizar el dato.
- 4 Notificar de inmediato ante residuos peligrosos o severidad alta.
- 5 Exponer indicadores operativos para el seguimiento de la entidad.

ALCANCE DE LA ENTREGA

Actor primario

Ciudadano residente

Actor secundario

Operador de aseo

Territorio piloto

Una localidad urbana

Flujo cubierto

Reporte → clasificación → agrupamiento → priorización → cierre

Fuera de alcance

Facturación, despacho de vehículos, comparendo ambiental

De una foto en la calle a una cuadrilla en ruta

El ciudadano toma una foto, la app resuelve la ubicación y envía el reporte en menos de un minuto. En la nube el reporte se clasifica, se agrupa con sus vecinos y se prioriza.



Arquitectura serverless en AWS

CLIENTE

App Flutter

Android · iOS

BORDE

Cognito

auth + grupos

API Gateway HTTP

JWT · throttling

CÓMPUTO

Lambda × 8

Python 3.12 · arm64

Un rol IAM por función

mínimo privilegio

DATOS

DynamoDB

single-table · 3 GSI

S3 + KMS

evidencias cifradas

ASÍNCRONO

EventBridge + SNS

alertas prioritarias

Rekognition

clasificación asistida

SQS DLQ

nada se pierde

Cada servicio responde a un rasgo del problema

RASGO DEL PROBLEMA

SERVICIO ELEGIDO

POR QUÉ

Demanda muy irregular: picos tras lluvias, casi nula de madrugada

Lambda + API Gateway

Escala a cero; sin capacidad ociosa

Consultas siempre por zona o por estado, nunca analíticas

DynamoDB single-table

Todo es Query por clave; nunca Scan

La evidencia pesa mucho más que el resto del reporte

S3 + URL prefirmada

La foto no pasa por el cómputo

Clasificar residuos sin dataset propio inicial

Rekognition

Línea base inmediata y reemplazable

Nuevos consumidores del evento a futuro

EventBridge

El productor no conoce a sus consumidores

Datos personales implícitos: ubicación y foto

KMS + IAM + TTL

Cifrado, mínimo privilegio, retención acotada

Descartados con razón documentada: contenedores en ECS/EKS (costo base injustificado), PostgreSQL + PostGIS (instancia siempre encendida y VPC innecesaria), Firebase (menor ejercicio de arquitectura).

Una sola tabla, cuatro preguntas, ningún Scan

En DynamoDB el modelo se diseña desde las consultas, no desde las entidades. Hay exactamente cuatro consultas en el producto y cada una tiene su clave.

Detalle de un reporte

Tabla principal

REPORT#<uuid>

¿Qué hay cerca de aquí?

GSI1 · geohash

GEO#<geohash6>

¿Qué está pendiente?

GSI2 · estado

STATUS#<estado>

¿De quién es esta foto?

GSI3 · disperso

EVID#<clave>

Geohash a dos precisiones

Se guarda con precisión 7 (~153 m) pero se particiona por los 6 primeros caracteres (~1,2 km): un barrio completo en una sola consulta, sin partición caliente.

Índice disperso

GSI3 solo indexa reportes con evidencia, con proyección KEYS_ONLY. Sin él, cada foto subida costaría un Scan de la tabla entera.

TTL de 540 días

Es a la vez decisión de privacidad y de costo: el reporte y su evidencia en S3 caducan juntos.

De reportes sueltos a puntos críticos priorizados

1

Prefiltrado por geohash

Se consultan la celda del punto y sus vecinas en GSI1. Acota el conjunto a decenas de reportes sin escanear la tabla.

2

Agrupamiento por densidad

Variante de DBSCAN sobre distancia de Haversine: cada reporte no asignado abre un grupo y absorbe a los que estén dentro del radio.

3

Umbral de existencia

Un grupo es punto crítico solo con 3 o más reportes. Evita que una queja aislada movilice una cuadrilla.

4

Priorización

Severidad acumulada ordena los grupos; la categoría dominante define el protocolo de manejo.

COMPORTAMIENTO VERIFICADO

4 reportes en 8 m

1 punto crítico · severidad 15 · prioridad Alta

Los mismos, ya atendidos

0 puntos: lo cerrado sale del análisis

Solo 2 reportes cercanos

0 puntos: no alcanza el umbral

3 cercanos + 3 a 4 km

2 puntos independientes

Radio pedido: 99.999 m

Se acota a 1000 m en el servidor

Seguridad y privacidad por diseño

Autenticación en el borde

API Gateway valida el JWT antes de invocar código propio. Una petición sin credenciales nunca ejecuta una Lambda.

Minimización de datos

No se pide nombre, cédula ni teléfono. La única identidad es el sub opaco de Cognito.

Mínimo privilegio real

Ocho roles IAM, uno por función. get_report solo lee; presign_evidence solo firma en un prefijo del bucket.

Seudonimización en logs

Se registra sha256(user_id) truncado: suficiente para correlacionar, insuficiente para identificar.

Cifrado extremo a extremo

TLS obligatorio por política de bucket; KMS con rotación anual en DynamoDB, S3, logs, SNS y SQS.

Sin fuga en los errores

Un fallo interno devuelve 500 genérico con request_id. Hay una prueba que lo verifica.

Deuda reconocida: sin verificación anti-fraude por EXIF, sin WAF, sin desenfoco automático de rostros y sin reproceso automático de la DLQ. Documentarlo es parte del rigor.

Prácticas de ingeniería que sostienen el resultado

112

Pruebas automatizadas

Corren sin credenciales de AWS y sin red

82 %

Cobertura total

Dominio y servicios al 100 %

8

Roles IAM

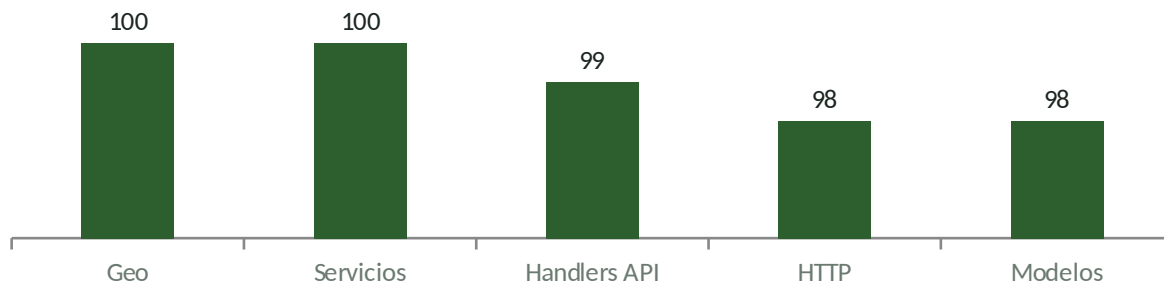
Uno por función, permisos declarados

0

Recursos creados a mano

Toda la infraestructura en Terraform

Cobertura por módulo del dominio (%)



Dependencias hacia el dominio

Handlers delgados sobre una capa de servicios que no conoce HTTP ni AWS.

Repositorio intercambiable

Un doble en memoria con la misma semántica permite probar sin nube.

CI en cada push

ruff, bandit, pytest con umbral de cobertura y terraform validate.

Despliegue con OIDC

GitHub asume un rol de AWS: cero llaves de larga vida almacenadas.

Demostrarla no cuesta nada; operarla, cinco dólares

Operación real de una localidad: 3.000 reportes y 30.000 consultas de mapa al mes, con foto promedio de 400 KB.



■ Rekognition ■ KMS ■ EventBridge · SNS · CloudWatch
■ DynamoDB ■ S3 ■ Lambda
■ API Gateway

0 USD

La demostración cabe en la capa gratuita de AWS

Operación real de una localidad: ≈ 4,85 USD / mes

El 62 % del costo es Rekognition

Un clasificador propio en el dispositivo eliminaría ese renglón: es el argumento económico de la investigación.

El resto escala a cero

Sin tráfico no hay factura: no hay instancias, ni NAT Gateway, ni capacidad reservada.

La clave KMS es opcional

Único cargo fijo del diseño (1 USD/mes). Desactivada por defecto; el dato sigue cifrado con claves de AWS.

De taller de curso a línea de investigación

Existe literatura sobre crowdsourcing ambiental y sobre clasificación automática de residuos, pero es escasa la evidencia sobre su combinación en contextos urbanos latinoamericanos.

FASE 1

Validación concurrente

Contrastar los puntos críticos derivados de reportes ciudadanos con el inventario oficial: precisión, exhaustividad y F1, con barrido de parámetros de agrupamiento.

FASE 2

Clasificador propio

Corpus anonimizado y etiquetado por doble anotador; MobileNetV3 ajustado frente a la línea base, medido en F1, latencia y costo.

FASE 3

Modelo predictivo

Rejilla de geohash y modelo de conteo para estimar reaparición a 7 días: pasar de recolección reactiva a preventiva.

La arquitectura actual ya habilita la investigación

- ✓ Evidencias en S3 con etiquetas de IA almacenadas: corpus en formación desde el primer reporte
- ✓ geohash + created_at indexados: la serie espacio-temporal ya existe
- ✓ Parámetros de agrupamiento configurables: los experimentos son reproducibles
- ✓ Publicación en EventBridge: agregar un consumidor de datos no toca el código que funciona

Qué se entrega

mobile/

App Flutter

Modelos, servicios (API, auth, ubicación, cola sin conexión), pantallas de reporte y mapa, tema accesible Material 3 y pruebas.

backend/

API en Python 3.12

Ocho handlers, capa de dominio y servicios, repositorios intercambiables, 112 pruebas y peticiones listas en api.http.

infra/

Terraform

DynamoDB, S3, Cognito, Lambda con roles de mínimo privilegio, API Gateway, EventBridge, SNS, DLQ, alarmas y tablero.

docs/

Documentación técnica

Diez documentos: análisis, arquitectura, modelo de datos, seguridad, ADR, pruebas, proyección, despliegue, API y guion de sustentación.

Repositorio público en GitHub

github.com/DiegoGamba/ecoruta

README con diagramas y guía de despliegue · CI con pruebas, análisis de seguridad y validación de infraestructura · Licencia MIT

EN RESUMEN

El conocimiento local del vecino, convertido en un inventario vivo y priorizado.

Problema real y delimitado

Puntos críticos dinámicos que los inventarios manuales no alcanzan

Ingeniería verificable

112 pruebas, roles de mínimo privilegio, infraestructura como código y CI

Arquitectura justificada

Cada servicio responde a un rasgo del problema, con alternativas descartadas por escrito

Camino investigativo abierto

Validación concurrente, corpus abierto y clasificador propio en el dispositivo